

REPUBLIQUE DU CAMEROUN

Paix--Travail--Patrie
MINISTERE DE L'ENSEIGNEMENT SUPERIEUR

REPUBLIC OF CAMEROON

Peace—Work—Fatherland
MINISTRY OF HIGHER EDUCATION



Université de Yaoundé I
Ecole nationale supérieure Polytechnique
de Yaoundé I



PROGRAMMATION PYTHON GTEL 3036 & GENIE LOGICIEL GTEL 3046:

Rapport de développement d'une Application Web de Gestion de Restaurant à partir de la Framework Django

Membres du Groupe	Matricules	Participation
VMESSAH DJOUMG IDA	23P343	20 40 36% 38% 40% 40% 33%
ENGAMBA ARMAND CHRISTIAN	25P882	20 40 35% 38% 40 % 40% 33%
NGOMANDJOLIE EKWE LARISSA	22P500	20 15 14% 10% 8% 8% 9%
CHINTOUO ARAFAT	22P507	20 0 2% 0% 2 % 2% 8%
JOHN KOLLE EPIE JABEA	23P473	20 5 13% 14% 8 % 8% 9%
BIKELLE FRITZ	21P151	8%

GROUPE 04

Encadreur : M. Mbietieu Amos Mb.

Année académique : 2025 – 2026

TABLE DES MATIÈRES

1. INTRODUCTION GENERALE	4
1.1 INTRODUCTION	4
2. Installation de l'environnement Python et Django.....	4
2.1 Prérequis système	5
2.2 Python et environnement virtuel.....	5
2.3 Installation et configuration de Django	5
Configuration de MySQL.....	6
2.4 Installation de MySQL	6
3. Mise en place du dépôt GitHub	6
4. Outils de gestion de projet.....	8
4.1 GanttProject.....	8
4.2 StarUML.....	9
5. Les diagrammes UML	10
5.1 La modélisation UML	10
Séquence 1 : Authentification de l'utilisateur.....	15
Séquence 2 : Prise de commande par le serveur.....	15
Séquence 3 : Gestion des recettes par le chef cuisinier	16
Séquence 4 : mise à jour du stock en cas d'alerte.....	16
6. Application au projet restaurant.....	23
7. ETUDE DU LANCEMENT.....	24
7.1 Contexte général.....	24
7.2 Problématique.....	24
7.3 Objectifs du projet	24
7.4 Structure du rapport.....	25
8 PRESENTATION DU DOMAINE METIER.....	26
9. ANALYSE DES BESOINS	27
10. CONCEPTION PRELIMINAIRE DE LA SOLUTION	32
11. PLANIFICATION ET ORGANISATION DU TRAVAIL.....	36
13. CONCLUSION ET PERSPECTIVES	43
10.1 Bilan du rapport de lancement.....	43
10.2 Impacts attendus	43
10.3 Perspectives d'évolution	43

TABLE DES FIGURES

Figure 1: Diagramme de Gantt	8
Figure 2: Plannification Gantt	9
Figure 3: Diagramme des cas d'utilisation.....	11
Figure 4: Diagramme de classes.....	13
Figure 5:Diagramme de sequence du module authentication	15
Figure 6:Diagramme de sequence du module commande.....	15
Figure 7:Diagramme de sequence du module recettes	16
Figure 8:Diagramme de sequence du module inventaire	16
Figure 9:Diagramme de sequence du cas d'utilisation passer une commande avec stock insuffisant	17
Figure 10:Diagramme de sequence du module tableau de bord.....	18
Figure 11:Diagramme de sequence du module commande cas d'utilisation annulation de commande.....	19
Figure 12: Diagramme de sequence du module recette cas de suppression de recette.....	20
Figure 13:Diagramme de sequence cas d'ajout d'un plat au catalogue.....	20
Figure 14:Diagramme de sequence du cas d'une modification de seuil.....	21
Figure 15:Diagramme de deploiement	22
Figure 16: exemple de suivi d'une connexion avec des comptes deja existants de la base de donnees	38

TABLE DES TABLEAUX

Table 1: stack technologique	4
Table 2: creation de l'environnement virtuel	5
Table 3: Tableau recapitulatif des commandes pour Github.....	7
Table 4: lecture dans StarUML.....	9
Table 5: Liste et role des diagrammes UML de ce projet.....	10
Table 6:Extrait des types de relations dans un diagramme de classes	12
Table 7: Tableau des acteurs.....	23
Table 8:Tableau des modules.....	23
Table 9: responsabilites des acteurs.....	26
Table 10: fichiers dedies du module intelligence artificielle	32
Table 11: presentation des couches du model MVT de Django	32
Table 12: presentation des modules de l'application.....	33
Table 13: objectif des diagrammes UML	36
Table 14: Structure modulaire Orientée Objet du projet.....	36
Table 15: planning Agile.....	37
Table 16: Recapitulatif des livrables	40
Table 17: difficultees rencontrees et solution apportees	41

1. INTRODUCTION GENERALE

1.1 INTRODUCTION

1.1.1 Contexte et objectifs

Dans un contexte comme le nôtre où la digitalisation avance à grand pas, où la gestion manuelle génère des pertes estimées à 20-30% dues aux erreurs de stock et commandes, la digitalisation accélérée de la restauration est une solution réelle. Notre objectif général est de développer une app Django-MySQL qui intègre 7 modules pour une traçabilité totale, aligné sur les standards industriels comme ceux de Toast ou Lightspeed. Cette approche POO via Django assure une scalabilité future, évitant les silos de données typiques des outils legacy.

Cette première partie du rapport documente la mise en place progressive de l'environnement de développement pour le projet Application de Gestion d'un Restaurant, réalisé dans le cadre du cours de Génie Logiciel et de programmation python. L'application cible la gestion intégrée d'un restaurant à travers **7 modules fonctionnels**.

1.1.2 Logiciels

Le tableau ci-dessous présente l'ensemble des outils et technologies retenus pour ce projet.

Table 1: stack technologique

Technologie	Version	Rôle
Python	3.12.3	Langage de programmation principal
Django	4.2.11	Framework web backend
MySQL	8.0	Système de gestion de base de données
Git / GitHub	2.53.0	Versionnage et collaboration en équipe
GanttProject	3.3.3316-1	Planification et gestion du projet
StarUML	7.0.0	Modélisation UML
Anthropic SDK	>= 0.25.0	Communication avec l'API Claude pour l'IA conversationnelle

Le stack (Python 3.12.3, Django 4.2.11, MySQL 8.0) est optimal pour la robustesse tandis que Django ORM protège contre les injections SQL, et que GitHub favorise la collaboration en feature-branch workflow.

2. Installation de l'environnement Python et Django

2.1 Prérequis système

Les commandes présentées dans ce rapport sont adaptées à un environnement **Ubuntu/Debian et Windows**, distribution utilisée sur les machines de l'équipe. Pour Windows, la commande d'activation de l'environnement virtuel diffère légèrement de celle d'Ubuntu.

2.2 Python et environnement virtuel

Un **environnement virtuel** est indispensable pour isoler les dépendances du projet de celles du système. Cela évite les conflits de versions entre projets et garantit la reproductibilité de l'installation sur toutes les machines de l'équipe. L'isolation via venv évite 90% des conflits de dépendances, crucial pour un projet de cette envergure autant pour Ubuntu que Windows. L'activation croisée (source env/bin/activate ou env\Scripts\activate) démontre une portabilité proactive.

Table 2: creation de l'environnement virtuel

Linux	Windows
Vérifier la version de Python installée python3 --version	Vérifier la version de Python installée python3 --version
# Créer le dossier du projet mkdir application_restaurant && cd application_restaurant	# Créer le dossier du projet mkdir application_restaurant && cd application_restaurant
# Créer l'environnement virtuel (ubuntu) python3 -m venv env	# Créer l'environnement virtuel virtuel (windows) python -m venv env
Activer l'environnement virtuel (Linux) source env/bin/activate	Activer l'environnement virtuel (Windows) env/Scripts/activate
L'invite de commande affiche désormais : (env) \$	L'invite de commande affiche désormais : (env)

Voici la liste des commandes successives nécessaires pour la création et de l'environnement de travail.

2.3 Installation et configuration de Django

Voici la liste des commandes successives nécessaires pour l'installation et la configuration de Django.

```
# Installer Django dans  
l'environnement virtuel pip  
install django  
# Vérifier l'installation  
django-admin --version  
#doit retourner 4.2.x  
# Créer le projet Django (le point '.' évite un dossier  
imbriqué) django-admin startproject config .
```

```
# Lancer le serveur de développement
python manage.py runserver
```

```
# Ouvrir http://127.0.0.1:8000 dans le navigateur
```

Après le lancement, l'interface d'accueil Django doit s'afficher dans le navigateur. Si c'est le cas, l'installation est réussie. Chaque module sera créé comme une **application Django indépendante** via la commande `python manage.py startapp nom_module`.

NB: /env ne doit pas être commit sur github

Configuration de MySQL

2.4 Installation de MySQL

Django supporte plusieurs systèmes de base de données. Pour ce projet, **MySQL 8.0** a été retenu pour ses performances, sa robustesse et son adéquation avec les besoins en production. Par défaut, Django utilise SQLite il faut explicitement reconfigurer la connexion.

```
# Installation de MySQL sur Ubuntu sudo apt update sudo apt install mysql-server -y
# Installation de MySQL sur Windows Télécharger MySQL Installer

Installer MySQL Server

# Vérifier le service MySQL sudo systemctl status mysql

# Sécuriser l'installation sudo mysql_secure_installation

# Se connecter à MySQL mysql -u root -p
```

3. Mise en place du dépôt GitHub

3.1 Initialisation Git et gitignore

Git est le système de versionnage local. GitHub est le serveur distant qui permet le travail collaboratif en équipe. Le dépôt doit être initialisé dès le début du projet, avant d'écrire la moindre ligne de code métier.

```
# Initialiser Git dans le dossier du projet git init

# Ajouter tous les fichiers au suivi git add .

# Premier commit git commit -m "Initial commit — structure Django + configuration"

# Relier au dépôt distant GitHub

git remote add origin https://github.com/votre-equipe/restaurant-app.git
```

git branch -M main git push -u origin main

3.2 Git en équipe

Le modèle recommandé pour un projet académique est le feature branch workflow: chaque membre travaille sur sa propre branche, puis fusionne dans la branche principale via une Pull Request revue par un autre membre.

Table 3: Tableau recapitulatif des commandes pour Github

Commande	Description
git status	Afficher l'état des fichiers modifiés
git add .	Indexer tous les fichiers modifiés
git commit -m "msg"	Créer un point de sauvegarde
git push origin main	Envoyer vers GitHub
git pull origin main	Récupérer depuis GitHub

4. Outils de gestion de projet

Comme outils nous avons utilisé comme cite précédemment Gantt project et StarUML. Gantt visualise les dépendances (ex: modélisation UML avant models.py), tandis que StarUML standardise les 4 diagrammes (Use Case, Classes, Séquence, Déploiement). Cohérence: ordre UML (Use Case → Classes → Séquence).

4.1 GanttProject

GanttProject est un logiciel libre de gestion de projet permettant de planifier les tâches, définir leurs durées, établir les dépendances entre elles et assigner des responsables. Le découpage temporel pour ce projet est tel que suit:

Gantt Chart

4

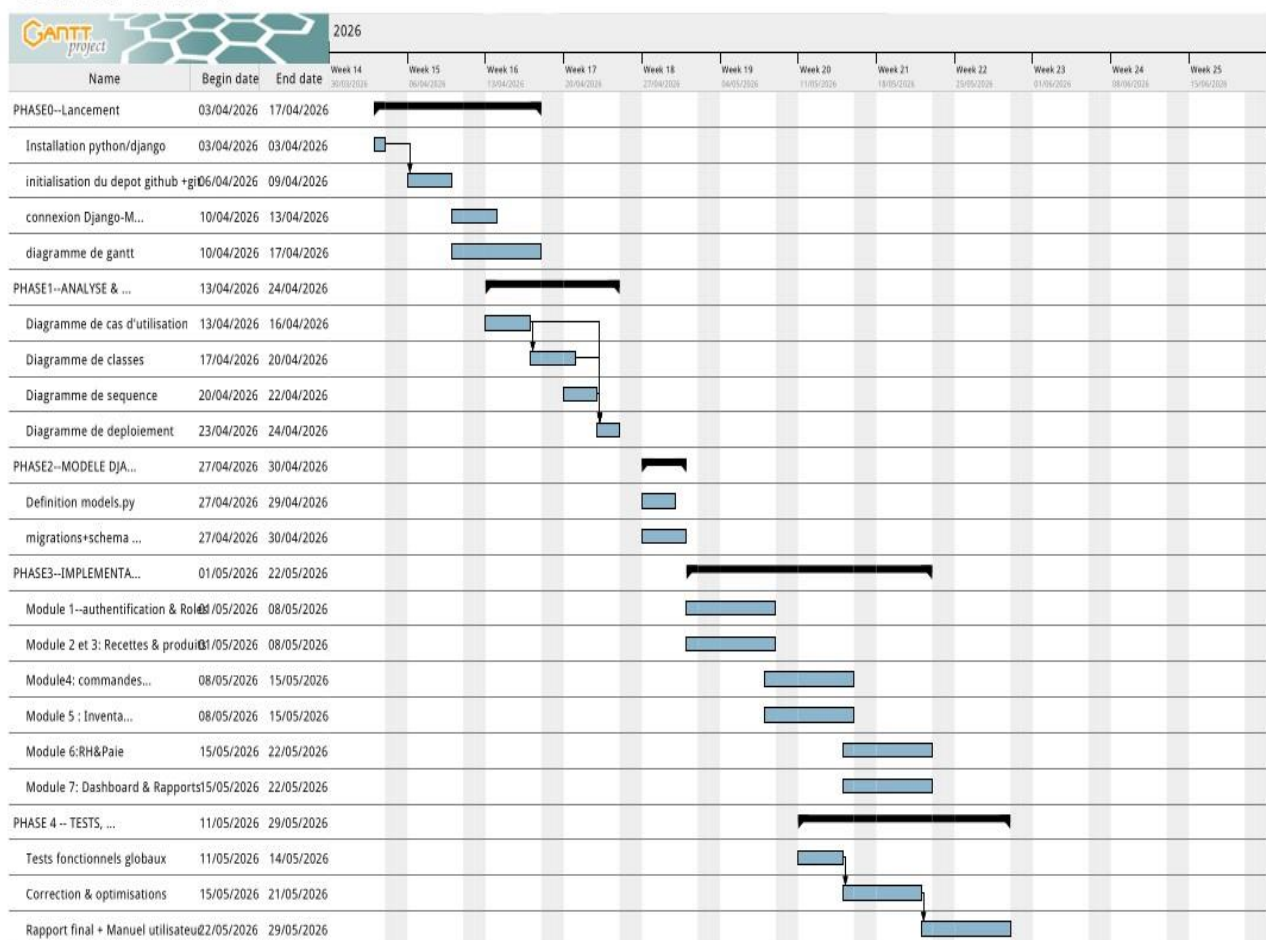


Figure 1: Diagramme de Gantt

Name	Begin date	End date
PHASE0--Lancement	03/04/2026	17/04/2026
Installation python/django	03/04/2026	03/04/2026
initialisation du depot github +git	06/04/2026	09/04/2026
connexion Django-MySQL operationelle	10/04/2026	13/04/2026
diagramme de gantt	10/04/2026	17/04/2026
PHASE1--ANALYSE & CONCEPTION_UML	13/04/2026	24/04/2026
Diagramme de cas d'utilisation	13/04/2026	16/04/2026
Diagramme de classes	17/04/2026	20/04/2026
Diagramme de sequence	20/04/2026	22/04/2026
Diagramme de deploiement	23/04/2026	24/04/2026
PHASE2--MODELE DJANGO ET MIGRATIONS	27/04/2026	30/04/2026
Definition models.py	27/04/2026	29/04/2026
migrations+schema BD MySQL operationnel	27/04/2026	30/04/2026
PHASE3--IMPLEMENTATION MODULES	01/05/2026	22/05/2026
Module 1--authentification & Roles	01/05/2026	08/05/2026
Module 2 et 3: Recettes & produits	01/05/2026	08/05/2026
Module4: commandes & facturation	08/05/2026	15/05/2026
Module 5 : Inventaire & Alertes stock	08/05/2026	15/05/2026
Module 6:RH&Paie	15/05/2026	22/05/2026
Module 7: Dashboard & Rapports	15/05/2026	22/05/2026
PHASE 4 -- TESTS, VALIDATION & FINALISATION	11/05/2026	29/05/2026
Tests fonctionnels globaux	11/05/2026	14/05/2026
Correction & optimisations	15/05/2026	21/05/2026
Rapport final + Manuel utilisateur	22/05/2026	29/05/2026

Figure 2: Plannification Gantt

4.2 StarUML

StarUML est un logiciel de modélisation UML multiplateforme, libre et largement utilisé dans l'industrie. **WinDesign** est son équivalent académique francophone. Les deux outils permettent de produire l'ensemble des diagrammes UML nécessaires au projet : cas d'utilisation, classes, séquence et déploiement. Dans le cadre de ce projet nous allons utiliser **StarUML**.

Table 4: lecture dans StarUML

Relation	Symbole	Signification
<<include>>	Flèche pointillée →	CU A inclut toujours CU B (B est obligatoire)
<<extend>>	Flèche pointillée →	CU B étend optionnellement CU A
Héritage	Flèche avec triangle	L'acteur Fils hérite des CU du Parent
Association	Trait plein	L'acteur interagit avec ce cas d'utilisation

C'est le tableau permet la lecture simple d'un diagramme des cas d'utilisation.

5. Les diagrammes UML

5.1 La modélisation UML

La modélisation UML précède toujours le développement. Elle permet de définir le périmètre fonctionnel, la structure des données, les interactions entre composants et l'architecture physique du système. L'ordre de construction recommandé est: Cas d'utilisation → Classes → Séquence → Déploiement

Table 5: Liste et role des diagrammes UML de ce projet

Diagramme	Outil	Objectif
Cas d'utilisation	StarUML	Identifier les acteurs (5 internes + 1 externe) et leurs interactions avec le système
Diagramme de classes	StarUML	Définir la structure des données, les attributs, méthodes et relations — traduit directement en models.py
Diagrammes de séquence	StarUML	Décrire les scénarios clés : authentification, prise de commande, gestion recettes, alerte stock
Diagramme de déploiement	StarUML	Représenter l'architecture physique client-serveur-SGBD de l'application

5.1.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation (Use Case) représente qui fait quoi dans le système. Il identifie les acteurs et les fonctionnalités qu'ils utilisent. Ce diagramme répond à la question « qui fait quoi? ».

Étapes de construction

1. Lister tous les acteurs qui interagissent avec le système.
2. Pour chaque acteur, lister toutes les actions qu'il peut effectuer — chaque action est un cas d'utilisation.
3. Identifier les relations: <<include>> pour les cas toujours inclus, <<extend>> pour les cas optionnels.
4. Regrouper les cas d'utilisation dans un rectangle « système » matérialisant la frontière de l'application.
5. Vérifier la cohérence: chaque acteur doit interagir avec au moins un cas d'utilisation.

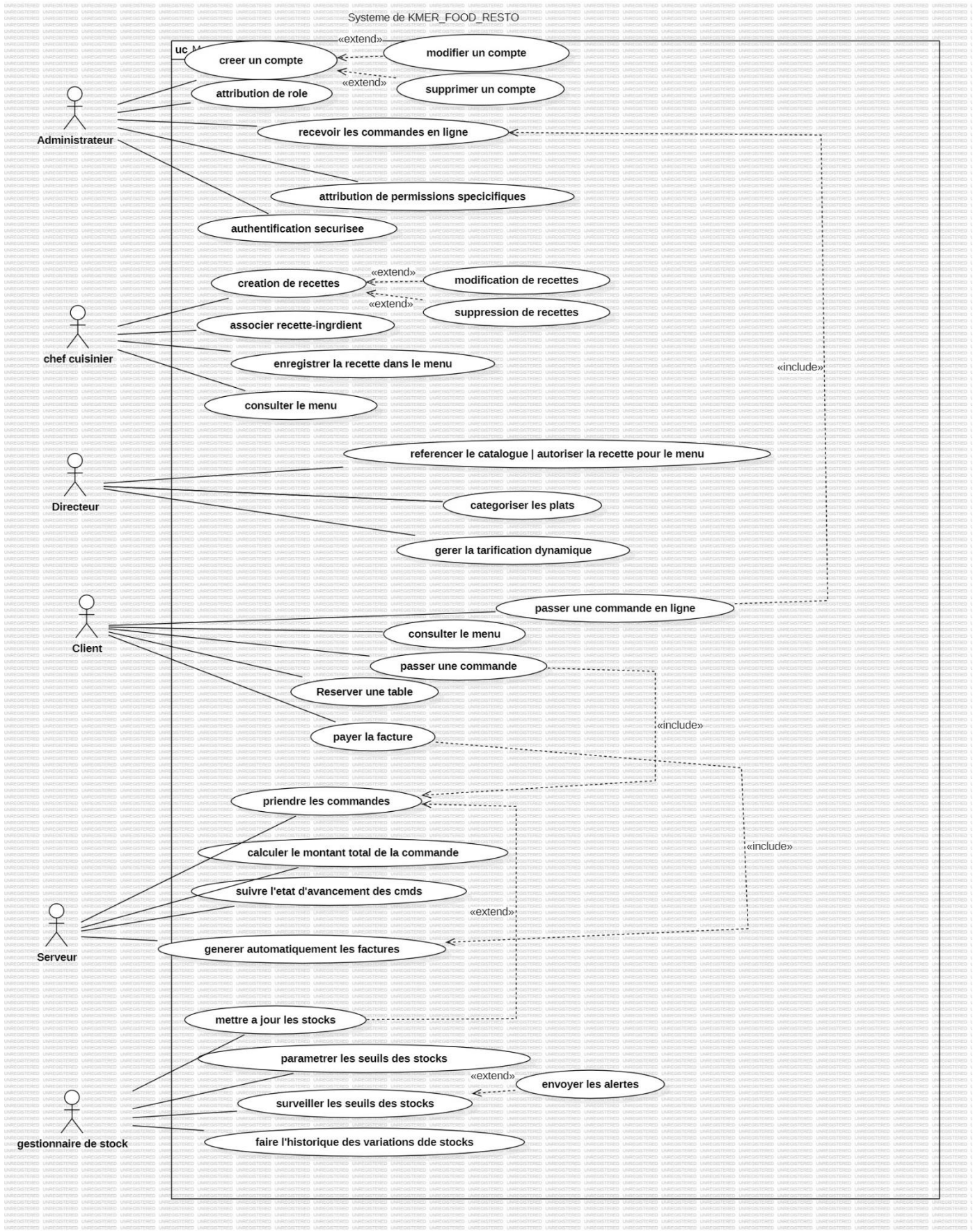


Figure 3: Diagramme des cas d'utilisation

Nous avons 6 acteurs intérieurs dont les actions sont représentés ci-dessus.

5.1.2 Diagramme de classes

Le diagramme de classes est le diagramme le plus important pour le développement. Il représente la structure des données, les attributs et méthodes de chaque classe, et les relations entre elles. Il est directement traduit en modèles Django et en tables MySQL.

Table 6:Extrait des types de relations dans un diagramme de classes

Relations	Symbole	Signification	Django
Association	→ (trait plein)	A utilise B	ForeignKey
Agrégation	◇ (losange vide)	B peut exister sans A	ForeignKey nullable
Composition	◆ (losange plein)	B ne peut exister sans A	ForeignKey + cascade
Héritage	△ (triangle)	B est un type spécialisé de A	Héritage de modèle
Realisation	△ (triangle) + ----	B realise A	ForeignKey

Il s'agit ici des types de relations et de symboles avec leurs significations permettant la lecture d'un diagramme de classe.

Structure d'une classe UML

NomDeLaClasse	← Nom (PascalCase)
- attribut1 : type	← - = privé \ = derive
+ attribut2 : type	← + = public
# attribut3 : type	← # = protégé
+ methode1() : retour	← Méthodes

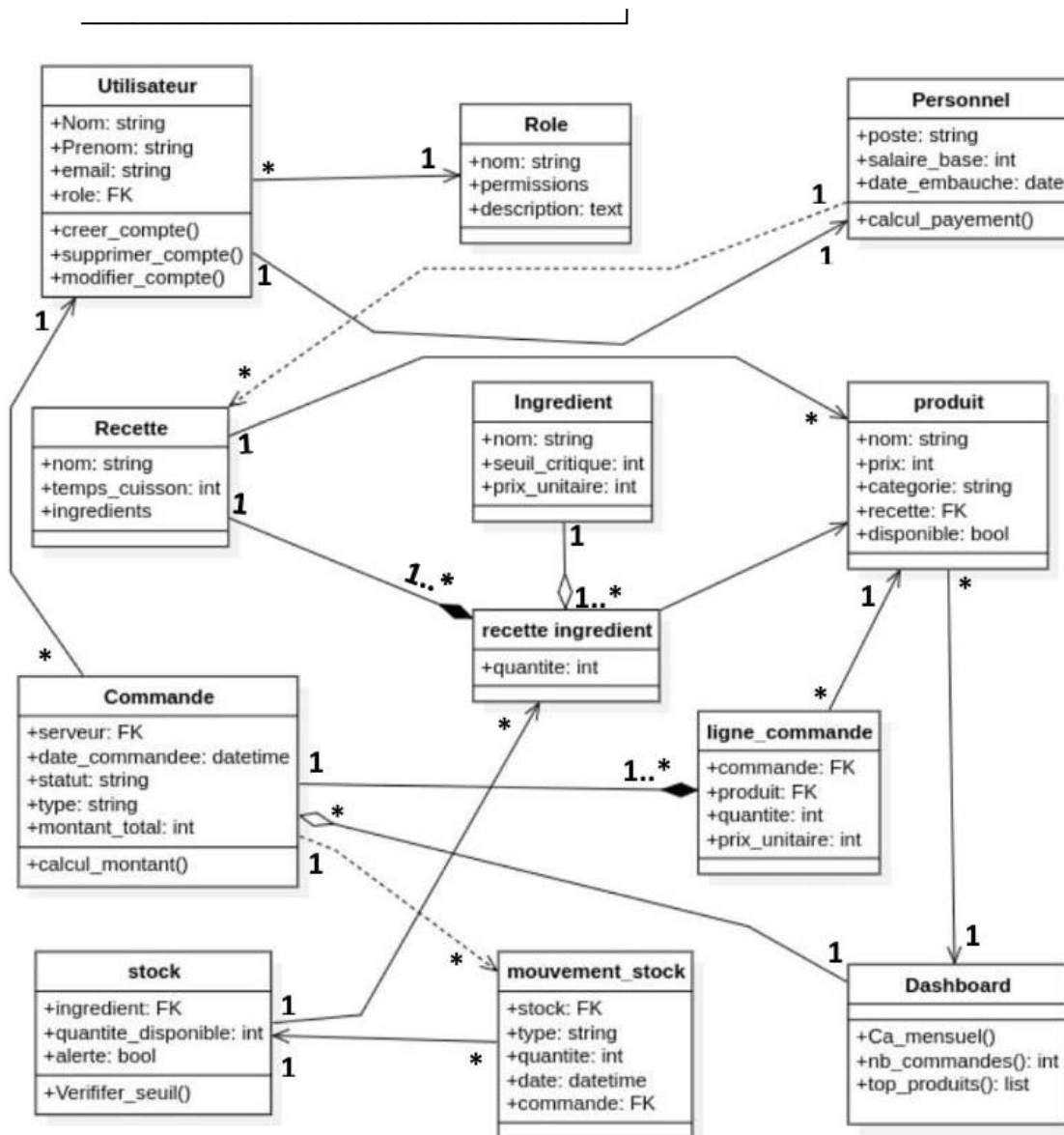


Figure 4: Diagramme de classes

Le diagramme couvre les 7 modules du projet.

Module Utilisateurs

Utilisateur gère les sessions, le hachage des mots de passe, etc. Il possède une ForeignKey vers Rôle, ce qui signifie qu'un utilisateur a un seul rôle parmi les 6 définis (Administrateur, Directeur, Chef cuisinier, etc.).

Rôle contient les permissions associées à chaque profil. C'est cette classe qui contrôle ce qu'un utilisateur peut faire dans l'application.

Module hr

Personnel est lié à Utilisateur par une relation OneToOne : chaque employé correspond exactement à un compte utilisateur. Il ajoute les données RH (poste, salaire de base, date d'embauche) et la méthode calculer_paie().

Module recette

Recette et **Ingredient** sont liés par une relation ManyToMany matérialisée par la classe intermédiaire RecetteIngredient, qui ajoute l'attribut quantité (la quantité de chaque ingrédient pour la recette). Une Recette est également liée au Chef (FK vers Personnel) qui en est responsable.

Modules produits

Produit représente les plats du menu. Il a deux ForeignKey importantes : une vers **Produit** et une vers Recette, un plat ne peut être vendu que s'il est associé à une recette valide.

Module Commandes

Commande est la classe centrale du flux de vente. Elle est liée au Serveur (FK vers

Utilisateur), contient l'état de la commande (en attente, en préparation, livrée, annulée), et possède la méthode calculer_montant(). Elle est en composition avec LigneCommande : une ligne de commande ne peut pas exister sans la commande parente. Chaque LigneCommande pointe vers un Produit avec une quantité et un prix unitaire.

Module Inventaire

Stock est lié à Ingredient (FK) et maintient la quantité disponible en temps réel. La méthode verifier_seuil () déclenche une alerte si la quantité tombe sous le seuil critique.

MouvementStock enregistre chaque entrée ou sortie de stock, avec un lien optionnel vers la Commande qui l'a déclenchée.

Module dashboard

Cette classe n'a pas de modèle Django propre : elle agrège les données des autres modules via des méthodes de calcul (ca_mensuel (), top_produits()...) et les expose uniquement au Directeur et à l'Administrateur.

5.1.3 Diagramme de séquence

Le diagramme de séquence représente le déroulement temporel des interactions entre objets pour un scénario précis. On produit un diagramme de séquence par cas d'utilisation complexe (ceux qui impliquent plusieurs échanges entre composants).

6. Participants: représentés par des rectangles en haut, avec une ligne de vie verticale pointillée.
7. Messages synchrones: flèches pleines horizontales (l'appelant attend la réponse).
8. Messages de retour: flèches pointillées (réponse renvoyée à l'appelant).

9. Ordre chronologique: les messages se lisent de haut en bas.

Séquence 1: Authentification de l'utilisateur

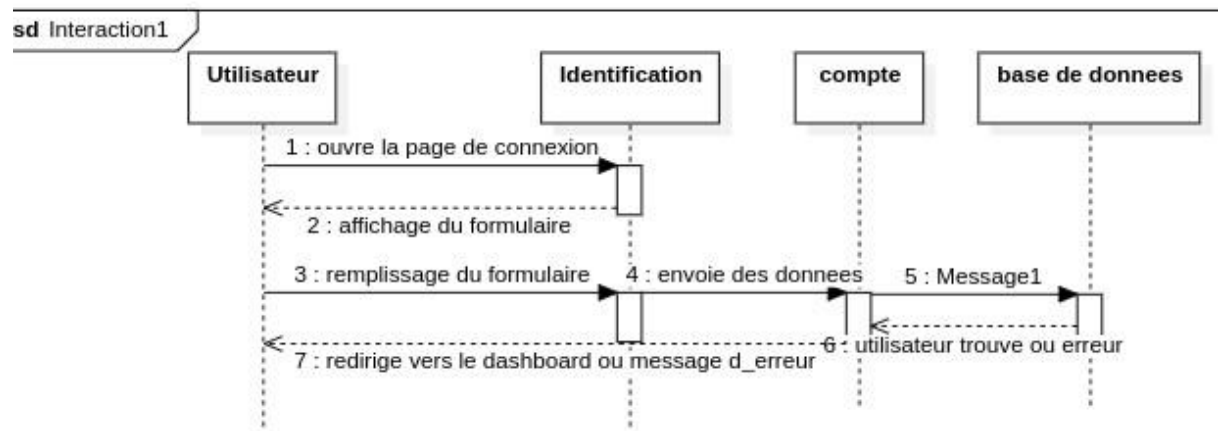


Figure 5:Diagramme de sequence du module authentication

Le serveur soumet ses identifiants → Django vérifie le mot de passe en base
→ récupère son rôle et ses permissions → redirige vers l'espace correspondant à son profil. En cas d'échec, un message d'erreur est retourné.

Séquence 2 : Prise de commande par le serveur

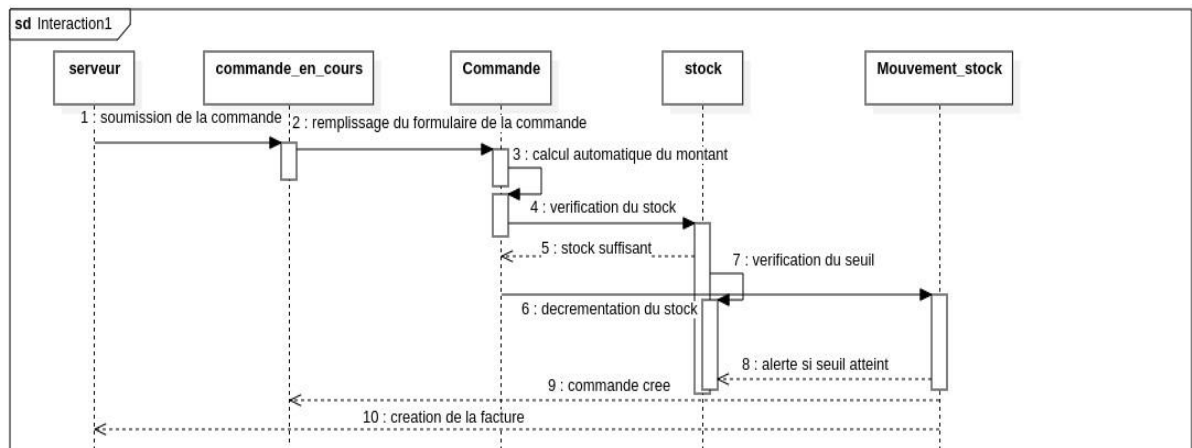


Figure 6:Diagramme de sequence du module commande

Le serveur soumet la commande → les lignes sont créées → le montant est calculé automatiquement → le stock est vérifié puis décrémenté → une alerte est levée si un seuil critique est atteint → la facture est retournée. Si le stock est insuffisant, la commande est rejetée.

Séquence 3 : Gestion des recettes par le chef cuisinier

Le chef ouvre le formulaire → Django charge la liste des ingrédients existants
→ le chef remplit nom, ingrédients et quantités → la recette et les lignes
RecetteIngredient sont insérées en base → redirection vers la liste.

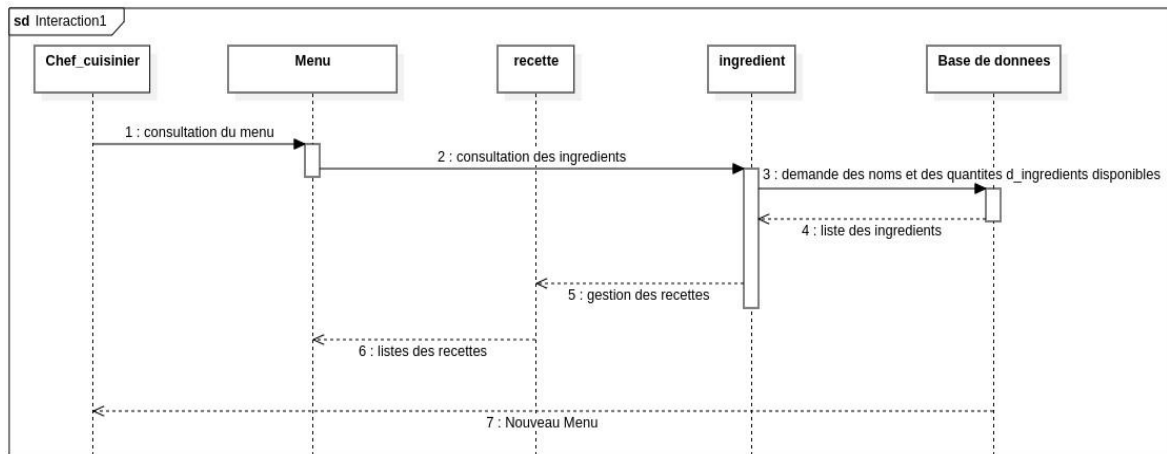


Figure 7:Diagramme de sequence du module recettes

Séquence 4 : mise à jour du stock en cas d'alerte

Lorsque la commande est validée, le stock est décrémenté, alors il y a vérification du seuil si le seuil a été traversé, une notification est envoyée au responsable du stock et l'alerte sur le stock reste active.

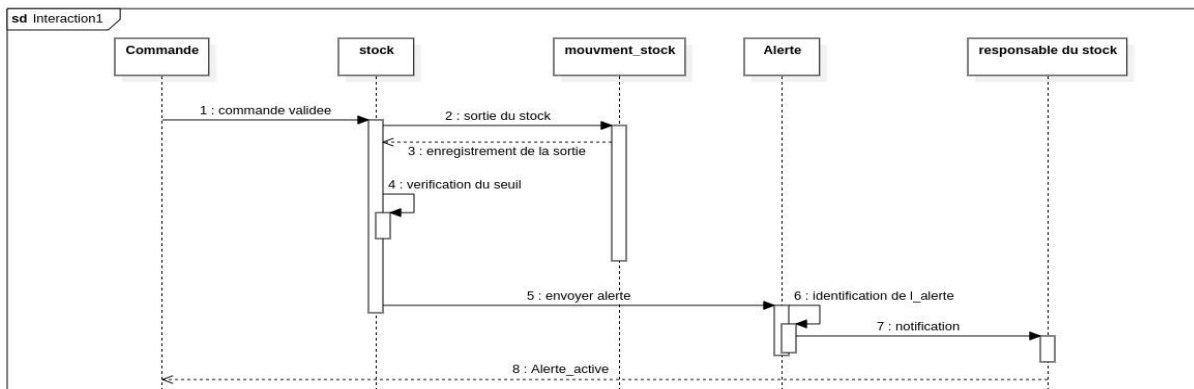


Figure 8:Diagramme de sequence du module inventaire

Séquence 5 : commande avec stock insuffisant :

Le serveur soumet la commande ; django crée la commande en état "en attente" ; pour chaque produits commandés, django récupère la recette associée ; pour chaque ingrédient de la recette, django vérifie les stock disponibles ; si un ingrédient est insuffisant alors la commande est annulée ; un message d'erreur "stock insuffisant pour ingrédient X" est retourné au serveur

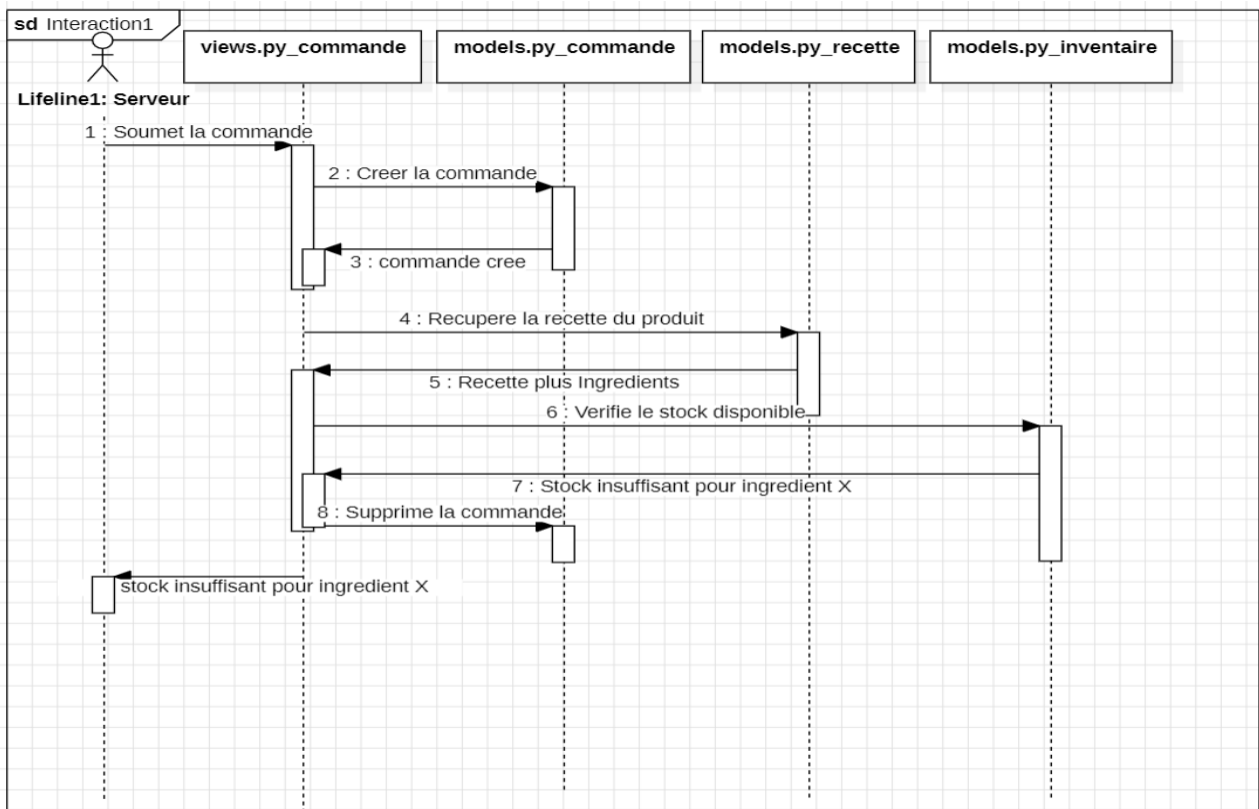


Figure 9: Diagramme de sequence du cas d'utilisation passer une commande avec stock insuffisant

Séquence 6 : consultation du dashboard

Le directeur accède a l'URL /dashboard ; django vérifie que l'utilisateur a le role directeur ; si role insuffisant l'accès sera refusé ; sinon django interroge les modules commande,produit,inventaire ,rh ; le template dashboard est rendu et retourné au directeur ; le directeur peut alors visualiser tous les indicateurs clés.

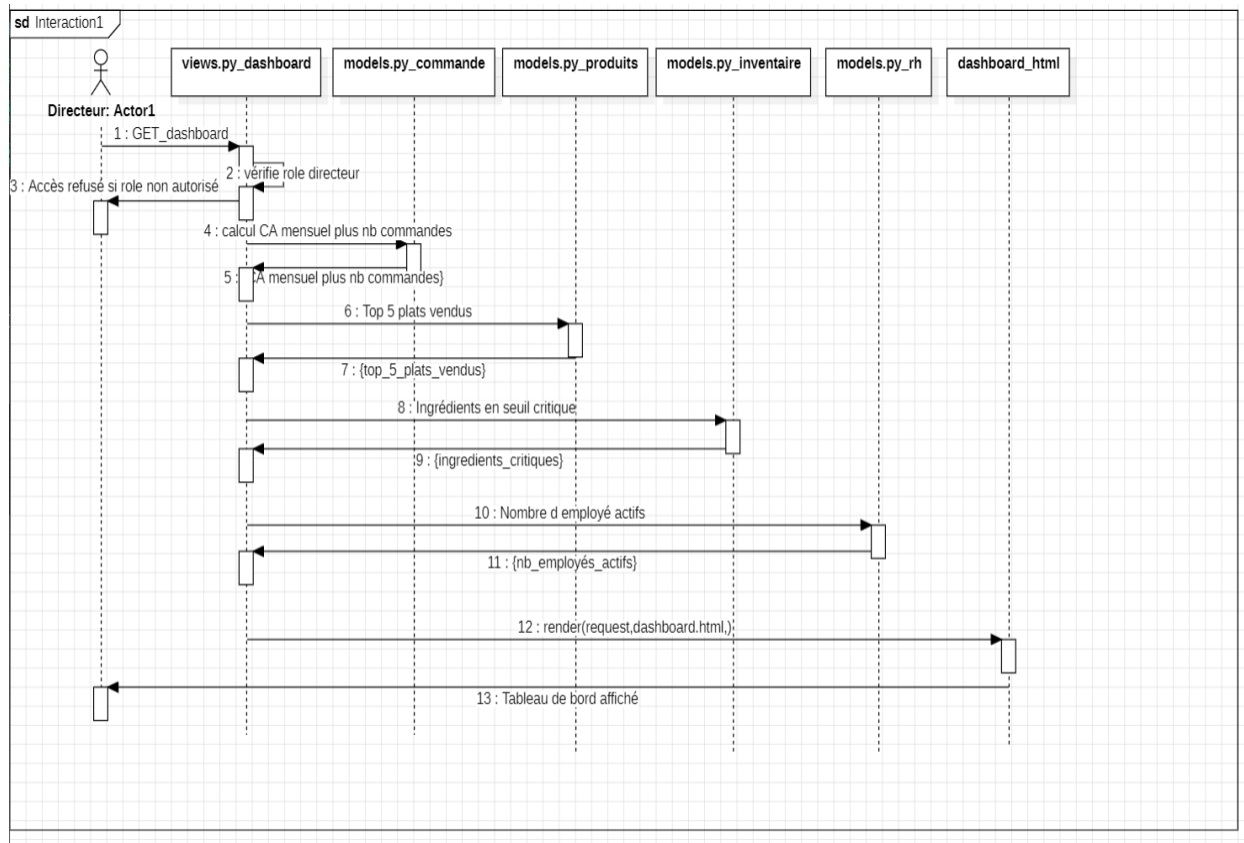


Figure 10:Diagramme de sequence du module tableau de bord

Séquence 7 : annulation d'une commande

Le serveur sélectionne un commande et clique sur annuler ;django récupère la commande puis vérifie son état ; si l'état est en préparation, "annulation impossible" sera retourné

Si non django restituera les quantités de chaque ingrédients dans le stock ; ensuite, l'état de la commande a annulé est mis a jour , la facture associée est ensuite invalidée et un message de confirmation est retourné au serveur .

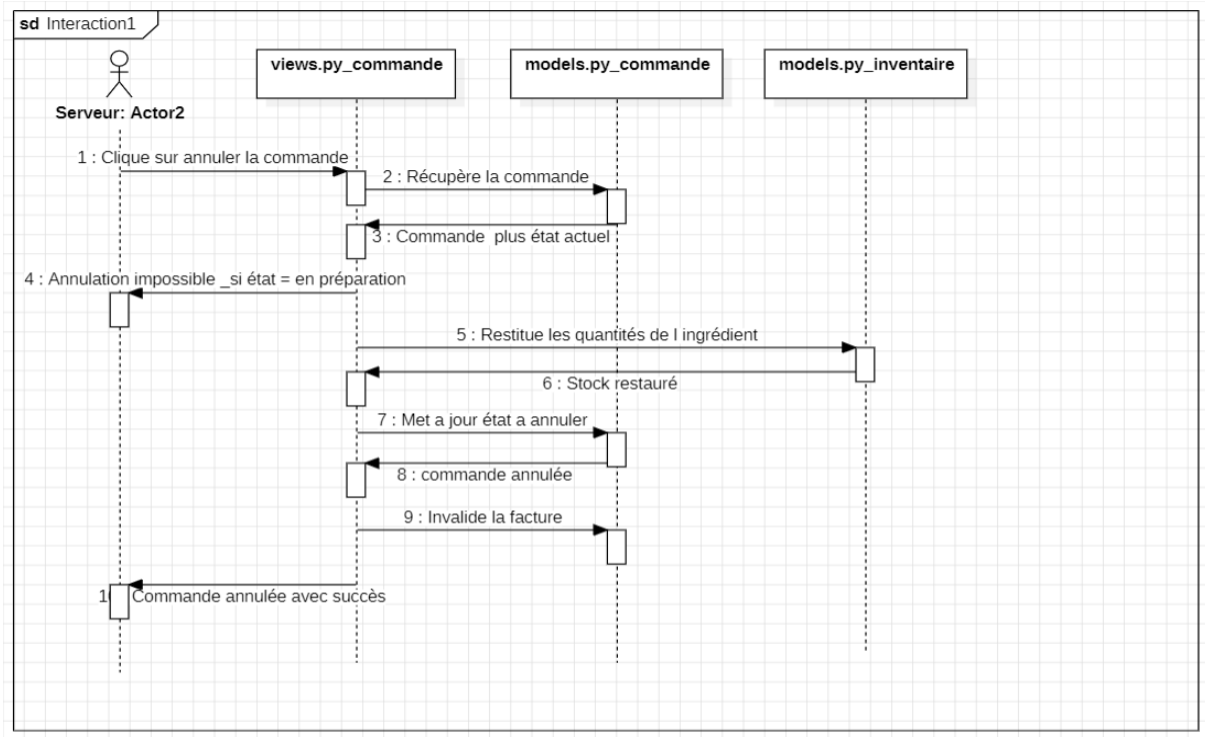


Figure 11: Diagramme de sequence du module commande cas d'utilisation annulation de commande

Séquence 8 : suppression d'une recette

Le chef sélectionne une recette et clique sur supprimer ; django recoit DELETE puis verifie si la rectte est associée a un plat actif dans le catalogue ; si oui la suppression est bloquée ; le message "recette utilisée par le plat X" s'affiche ; si non django supprime d'abord toutes les lignes RectteIngredient associées ; django supprime ensuite la recette puis le message "recette supprimée avec succès" est retourné au chef

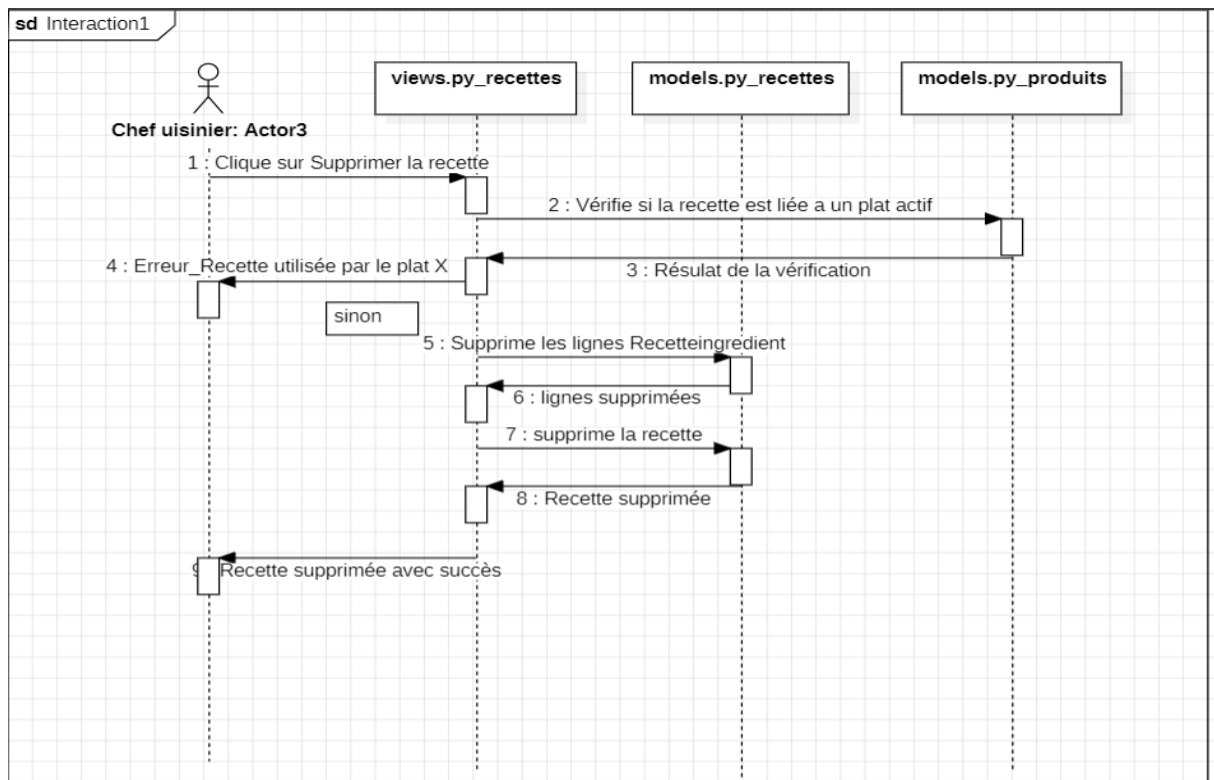


Figure 12: Diagramme de sequence du module recette cas de suppression de recette

Séquence 9 : Ajout d'un plat au catalogue

Le chef accède au formulaire d'ajout de plat ; django charge la liste des recettes disponibles et des catégories ; le chef remplit le formulaire : nom,description,prix,... puis il soumet le formulaire ensuite django valide les données ; django crée ensuite le plat en base et le message "plat ajouté avec succès" est affiché

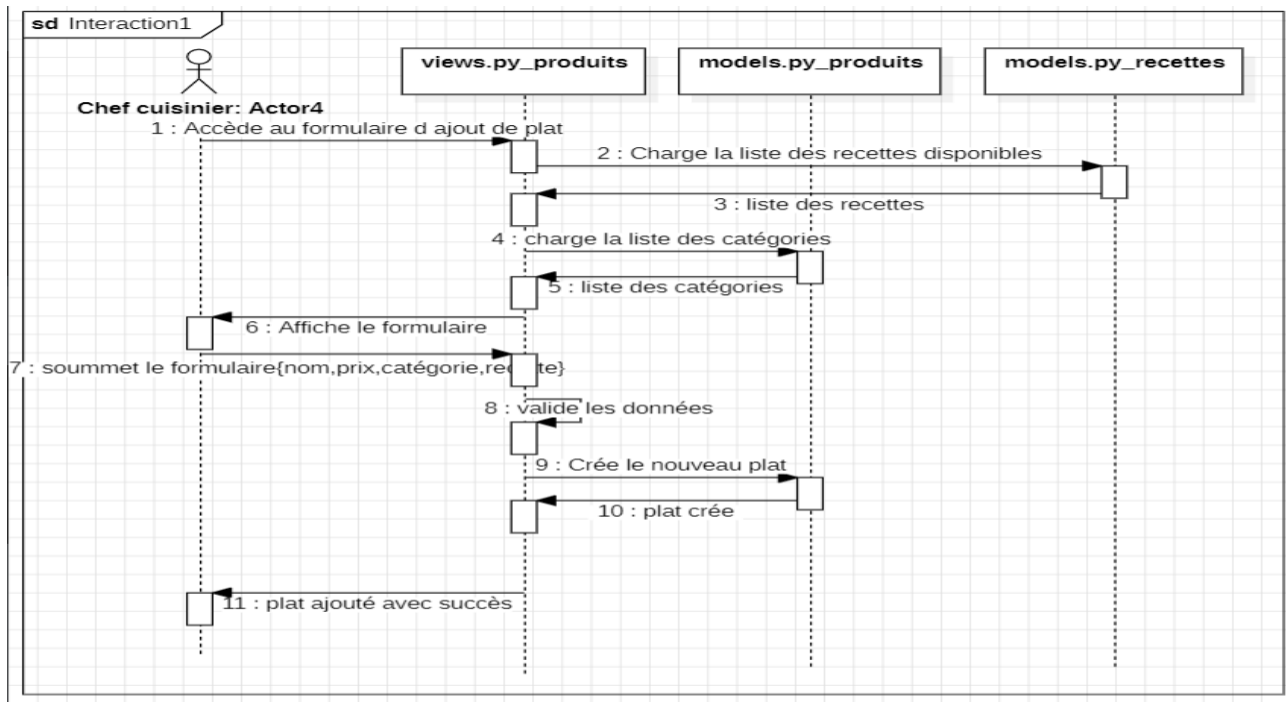


Figure 13:Diagramme de sequence cas d'ajout d'un plat au catalogue

Séquence 10 : modification du seuil critique d'un ingrédient

Le gestionnaire accède a la liste des ingrédients ; django charge la liste avec les seuils actuels ; le gestionnaire sélectionne un ingrédient et modifie son seuil critique puis il soumet le formulaire ; django valide que le seuil est un nombre positif, puis il met a jour le seuil ; le message "seuil mis a jour avec succès" est ensuite retourné au gestionnaire

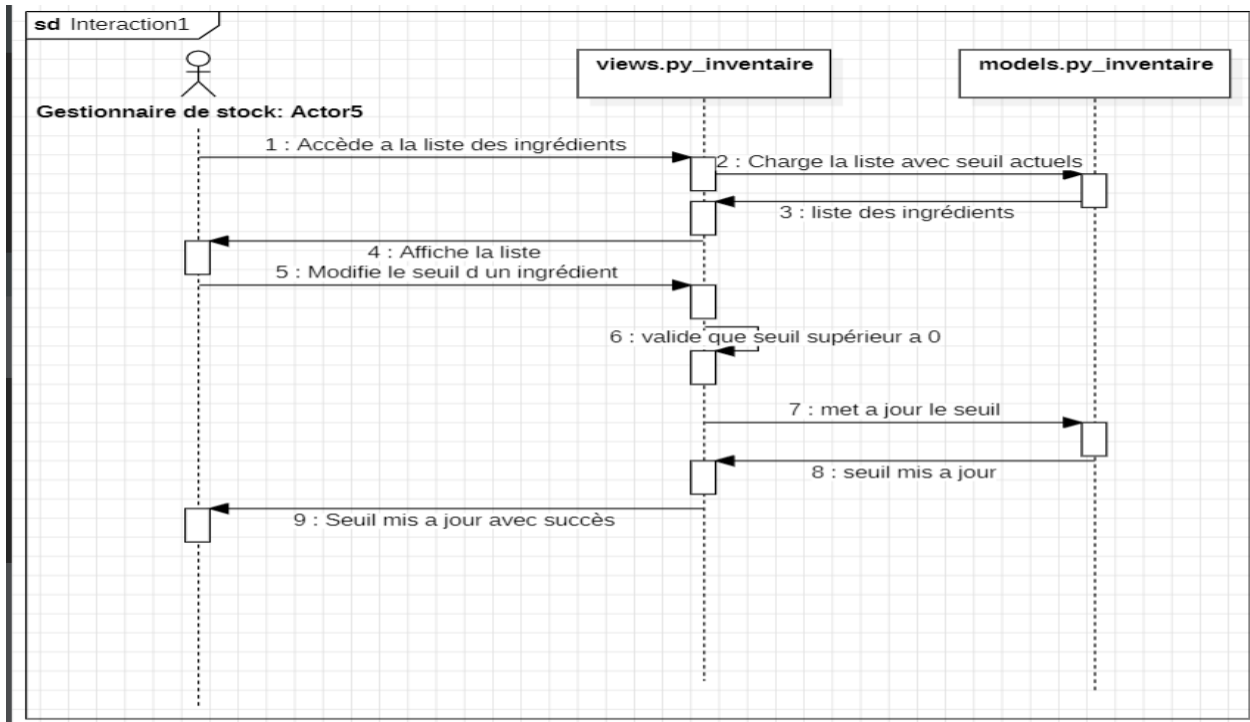


Figure 14:Diagramme de sequence du cas d'une modification de seuil

5.1.4 Diagramme de déploiement

Le diagramme de déploiement représente l'architecture physique du système. Pour ce projet, l'architecture cible est :

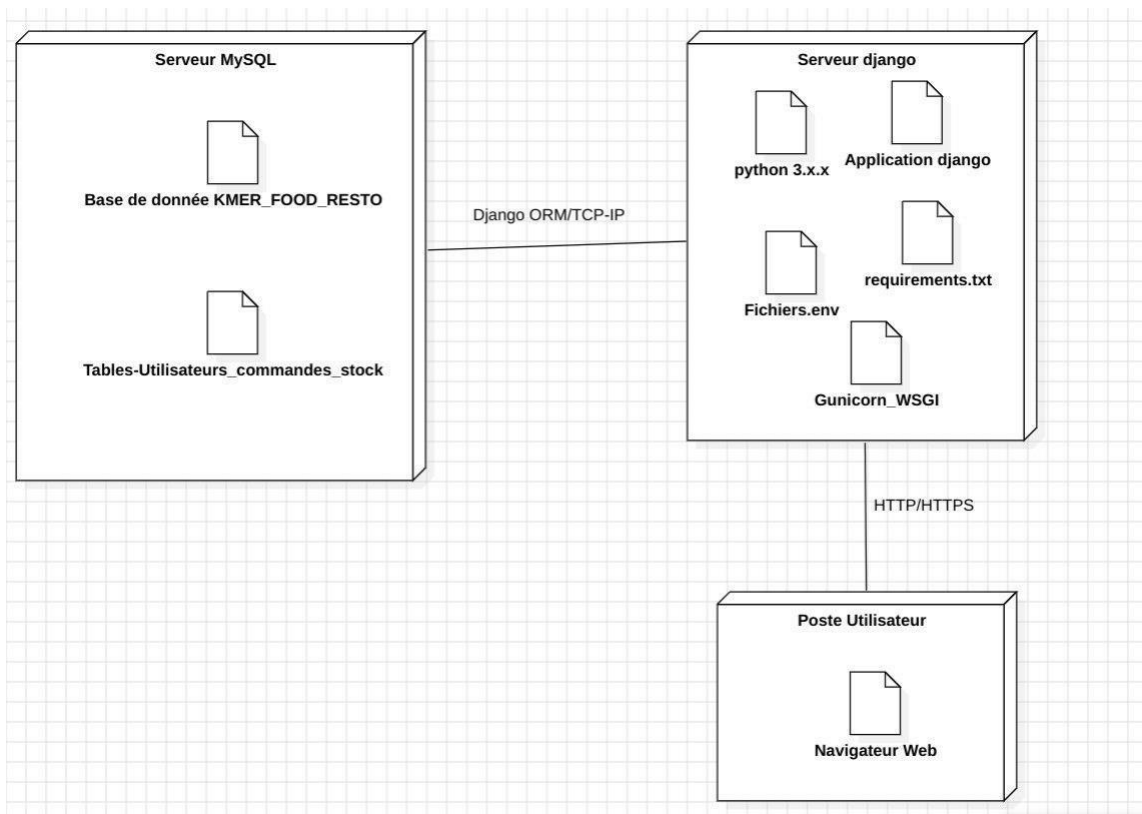


Figure 15:Diagramme de deploiement

6. Application au projet restaurant

6.1 Acteurs identifiés

Table 7: Tableau des acteurs

Acteur	Niveau d'accès	Modules concernés
Responsable du stock	Complet	Tous les modules + tableau de bord
Serveur	Partiel	Commandes, menu
Cuisinier	Consultation	Commandes (en attente), recettes
Responsable RH	Partiel	Ressources humaines, présences
Administrateur	Système	Utilisateurs, droits, paramètres
Client	Partiel	Commandes, authentification
Directeur	Totale	Tous les modules + tableau de bord

Dans le cadre de ce projet voici les acteurs que nous avons pu identifier jusqu'à présent.

6.2 Modules et applications Django

Table 8: Tableau des modules

Module	App Django	Modèles principaux
Utilisateurs/authentification	accounts	Utilisateur, Rôle, Permission
Recettes	Recettes	Recette, Ingrédient, StepRecette
Commandes	Commandes	Commande, LigneCommande, Table
Inventaire	Inventaire	Article, MouvementStock, Fournisseur
Ressources humaines	rh	Employé, Présence, Contrat, Salaire
Tableau de bord	dashboard	(agrégation des autres modules)
Reporting	Rapports	Rapport, ExportConfig
Assisstant IA	ia	(sans Modele : appels API)

Voici une liste des modules que nous allons créer tout au long de ce projet.

7. ETUDE DU LANCEMENT

7.1 Contexte général

Dans un environnement économique marqué par la digitalisation accélérée des services, le secteur de la restauration constitue un terrain stratégique pour l'innovation technologique. La transformation numérique n'est plus une option pour les entreprises de ce secteur, mais une nécessité imposée par la concurrence, les exigences clients et les impératifs de rentabilité.

Aujourd'hui, un restaurant moderne ne peut plus se limiter à une gestion manuelle de ses recettes, de ses commandes et de son inventaire. Il doit impérativement optimiser ses processus, assurer la traçabilité de ses opérations, améliorer l'expérience client et disposer d'indicateurs financiers en temps réel.

7.2 Problématique

La gestion manuelle d'un restaurant engendre plusieurs problèmes majeurs :

- Des erreurs humaines fréquentes dans la gestion des commandes et des stocks ;
- Un manque de visibilité sur les performances financières en temps réel ;
- Une difficulté à coordonner les différents acteurs (chefs, serveurs, gestionnaire de stock) ;
- Une absence de traçabilité sur les opérations critiques ;
- Des pertes économiques dues à une mauvaise gestion des approvisionnements. La mise en place d'un système d'information dédié permet de répondre à l'ensemble de ces défis de manière structurée et pérenne.

7.3 Objectifs du projet

Objectif général

Développer une application web complète de gestion de restaurant, basée sur le framework Django (Python), connectée à une base de données MySQL existante, et couvrant l'ensemble des besoins opérationnels et fonctionnels du domaine.

Objectifs spécifiques

1. Implémenter les modèles Django correspondant au schéma SQL existant ;
2. Configurer MySQL comme SGBD principal via Django ORM ;
3. Développer une interface d'administration personnalisée par rôle ;
4. Concevoir des interfaces utilisateurs ergonomiques et intuitives ;
5. Mettre en œuvre les règles métiers propres à chaque module fonctionnel

6. Générer des rapports financiers dynamiques (chiffre d'affaires, dépenses) ;
7. Garantir la sécurité des données et des accès utilisateurs.

7.4 Structure du rapport

Cette deuxième partie du rapport de lancement est organisé comme suit : après cette introduction, nous présentons le domaine métier de la restauration afin d'en comprendre les enjeux. Nous procédons ensuite à une analyse détaillée des besoins fonctionnels et non fonctionnels. La section suivante expose la conception préliminaire de la solution technique. Enfin, nous décrivons la planification du projet et la répartition des tâches au sein du groupe.

8 PRESENTATION DU DOMAINE METIER

8.1 Fonctionnement général d'un restaurant

Un restaurant est une entreprise de services dont l'activité principale consiste à proposer des repas préparés à une clientèle en échange d'une contrepartie financière. Son fonctionnement repose sur l'interaction étroite de plusieurs processus : la production culinaire, le service en salle, la gestion des approvisionnements, la gestion du personnel et le pilotage financier.

Ces processus impliquent des acteurs aux rôles bien définis qui doivent collaborer efficacement pour assurer la qualité du service et la viabilité économique de l'établissement. La numérisation de ces processus constitue le cœur du projet KMER FOOD.

8.2 Les acteurs du système

Le système d'information d'un restaurant met en jeu plusieurs profils d'utilisateurs, chacun disposant d'attributions précises:

Table 9: responsabilites des acteurs

Acteur	Responsabilités
Administrateur	Gestion globale du système, création des comptes, supervision générale.
Directeur	Supervision des performances financières, accès aux tableaux de bord et rapports.
Chef cuisinier	Gestion des recettes, des ingrédients et de la production en cuisine.
Cuisinier	Exécution des commandes de préparation selon les recettes définies.
Serveur	Prise des commandes clients, génération des factures et suivi des livraisons.
Gestionnaire de stock	Suivi de l'inventaire, gestion des alertes de seuil critique et des réapprovisionnements.
Client	Consultation du menu, passation de commandes (sur place ou en livraison), authentification.

8.3 Les processus métiers clés

Les principaux processus identifiés dans le domaine de la restauration, et qui serviront de base à la conception de l'application, sont les suivants :

a) Processus de commande

Il couvre l'ensemble du cycle de vie d'une commande client, depuis sa création par le serveur jusqu'à la génération de la facture. Une commande peut être passée sur place ou en livraison. Elle est associée à des produits du catalogue, et son montant est calculé automatiquement.

b) Processus de production culinaire

Chaque plat du menu est associé à une recette décrivant les ingrédients nécessaires, leurs quantités et le temps de préparation. Le chef cuisinier responsable de chaque recette est identifié dans le système.

c) Processus de gestion de l'inventaire

Le stock d'ingrédients est automatiquement mis à jour à chaque commande passée. Des alertes sont déclenchées lorsque le niveau d'un ingrédient passe en dessous d'un seuil critique défini, permettant des réapprovisionnements proactifs.

d) Processus de gestion des ressources humaines

Le système maintient les dossiers des employés, leurs affectations aux différents postes et le calcul de leur paie. Ces informations permettent une gestion administrative rigoureuse du personnel.

e) Processus de pilotage financier

Le tableau de bord de direction agrège les indicateurs clés: chiffre d'affaires mensuel, nombre de commandes, produits les plus vendus et dépenses d'approvisionnement. Ces données orientent les décisions stratégiques de l'établissement.

9. ANALYSE DES BESOINS

9.1 Besoins fonctionnels

L'application couvre sept modules fonctionnels distincts définis par le cahier des charges. Pour chaque module, les acteurs concernés, les fonctionnalités attendues et les règles métiers sont précisés.

Pour le bon fonctionnement des modules et la bonne cohésion de la base de données avec l'application les informations de la base de données ont d'abord été importées dans l'application.

Module 1 — Gestion des utilisateurs

Acteurs concernés: *Administrateur*

Fonctionnalités:

- Création, modification et suppression de comptes utilisateurs;
- Authentification sécurisée par identifiant et mot de passe;
- Attribution de rôles parmi les six profils définis: Administrateur, Directeur, Chef cuisinier, Cuisinier, Serveur, Gestionnaire de stock;
- Attribution de permissions spécifiques selon le rôle.

Règles métiers :

- Un utilisateur ne peut accéder qu'aux fonctionnalités correspondant à son rôle ;
- Seul l'Administrateur peut créer ou supprimer des comptes ;

Module 2 — Gestion des recettes

Acteurs concernés : *Chef cuisinier, Cuisinier*

Fonctionnalités :

- Création, modification et suppression de recettes ;
- Association recette-ingrédient avec les quantités requises ;
- Enregistrement du temps de cuisson et du chef responsable ;
- Consultation du catalogue de recettes disponibles.

Règles métiers :

- Une recette doit obligatoirement être associée à au moins un ingrédient ;
- Seul le Chef cuisinier peut créer ou modifier une recette ;
- La suppression d'une recette est impossible si elle est associée à un plat actif du catalogue.

Module 3 — Gestion des produits

Acteurs concernés : *Chef cuisinier, Directeur*

Fonctionnalités :

- Référencement du catalogue de plats avec descriptions et photos;
- Catégorisation des plats (entrées, plats principaux, desserts, boissons);
- Gestion de la tarification dynamique.

Règles métiers :

- Tout plat doit appartenir à une catégorie;
- Le prix d'un plat ne peut pas être négatif ou nul;
- Un plat ne peut être proposé à la vente que s'il est associé à une recette valide.

Module 4 — Gestion des commandes

Acteurs concernés: *Serveur, Client*

Fonctionnalités:

- Prise de commandes sur place et en livraison;
- Calcul automatique du montant total de la commande ;
- Suivi de l'état d'avancement des commandes (en attente, en préparation, livrée, annulée) ;
- Génération automatique des factures.

Règles métiers :

- Une commande doit contenir au moins un produit ;
- Le montant total est calculé automatiquement à partir des prix unitaires et des quantités ;
- Une commande ne peut être annulée que si elle est encore à l'état "en attente" ;
- Toute commande validée déclenche automatiquement une mise à jour du stock.

Module 5 — Gestion de l'inventaire

Acteurs concernés : *Gestionnaire de stock, Administrateur*

Fonctionnalités :

- Mise à jour automatique des stocks après chaque commande validée ;
- Paramétrage et surveillance des seuils critiques de stock ;
- Envoi d'alertes en cas de rupture imminente ;
- Historique des variations de stock.

Règles métiers :

- Lorsqu'un ingrédient atteint son seuil critique, une alerte est automatiquement générée ;
- Aucune commande ne peut être validée si un ingrédient nécessaire est en rupture de stock ;
- Seul le Gestionnaire de stock peut modifier les seuils critiques.

Module 6 — Gestion des ressources humaines

Acteurs concernés : *Administrateur, Directeur*

Fonctionnalités :

- Gestion des dossiers des employés (informations personnelles, poste, contrat);
- Affectation des employés aux différents services ;
- Calcul automatisé de la paie.

Règles métiers :

- Tout employé doit être associé à un rôle défini dans le système ;
- Le calcul de la paie est basé sur le poste occupé et le nombre de jours travaillés
- Seul l'Administrateur peut modifier les informations contractuelles d'un employé.

Module 7 — Tableau de bord (Dashboard)

Acteurs concernés : *Directeur, Administrateur*

Fonctionnalités :

- Affichage du chiffre d'affaires mensuel ;
- Statistiques sur le nombre de commandes traitées ;
- Classement des produits les plus vendus ;
- Suivi des dépenses d'approvisionnement.

Règles métiers :

- Les données du tableau de bord sont actualisées en temps réel ;
- Seuls le Directeur et l'Administrateur ont accès au tableau de bord complet ;
- Les rapports financiers peuvent être exportés au format PDF.

9.2 Besoins non fonctionnels

Au-delà des fonctionnalités métiers, l'application doit satisfaire des exigences de qualité transversales, regroupées selon les critères suivants :

Table 10: besoins fonctionnels

Exigence	Description	Critère de satisfaction
Sécurité	Protection CSRF activée sur tous les formulaires POST ; protection contre les injections SQL via l'ORM Django ; hachage des mots de passe (PBKDF2) ; contrôle d'accès par rôle.	Aucune faille d'injection ; sessions sécurisées.
Ergonomie	Interface intuitive, responsive et adaptée au profil de chaque utilisateur. Chaque rôle dispose d'un espace de travail personnalisé.	Prise en main sans formation préalable.
Performance	Temps de réponse optimisé pour les opérations courantes, grâce à une utilisation judicieuse des requêtes ORM et la mise en cache.	Temps de réponse < 2 secondes.
Traçabilité	Journalisation (logging) des actions critiques : connexions, modifications sensibles (prix, stocks, paie), suppressions d'enregistrements.	Historique complet consultable par l'Administrateur.
Maintenabilité	Code structuré, commenté et organisé selon les bonnes pratiques Django. Séparation claire des responsabilités par application.	Code lisible et documenté.
Versionning	Utilisation obligatoire de GitHub. Commits réguliers et documentés par chaque membre du groupe.	Historique Git traçable et contributions visibles.

Ce tableau présente l'ensemble des exigences non fonctionnelles de l'application ainsi que leur description c'est-à-dire ce en quoi les termes consistent dans le cadre de notre projet avec les critères de satisfaction de ces exigences.

9.3 MODULE ADDITIONNEL ASSISTANT INTELLIGENT “KEMI”

Contexte et justification

Au cours du développement de l'application, il est apparu que la simple digitalisation des processus métiers, bien que nécessaire, ne suffisait pas à différencier pleinement la solution KMER FOOD des outils de gestion classiques. C'est pourquoi le groupe a décidé d'intégrer, en complément du cahier des charges initial, un module d'intelligence artificielle conversationnelle. Ce module constitue une valeur ajoutée significative, alignée sur les tendances actuelles de la restauration numérique.

Description du module

Le module IA repose sur l'API Anthropic (modèle Claude Sonnet), intégrée directement dans le backend Django via le SDK officiel. Il se matérialise sous la forme d'un assistant conversationnel nommé **KEMI** (*Kmer Eating and Menu Intelligence*), accessible depuis l'interface de l'application.

KEMI opère selon deux modes distincts selon le profil de l'utilisateur connecté :

Mode Client Assistant nutrition et recommandation

Côté client, KEMI se présente comme une conseillère culinaire personnelle du restaurant. Elle est accessible depuis toutes les pages publiques de l'application via un widget de chat flottant. Ses fonctionnalités sont les suivantes :

- Conseil nutritionnel personnalisé : l'assistant adapte ses recommandations en fonction du régime alimentaire du client (diabète, végétarisme, allergies, intolérance au gluten, etc.) ;
- Recommandation de plats : KEMI oriente le client vers les plats du menu camerounais les mieux adaptés à son profil et à ses préférences du moment ;
- Accompagnement interactif : le client peut décrire ses envies en langage naturel et recevoir des suggestions instantanées.

Il est important de noter que dans ce mode, KEMI ne révèle jamais à l'utilisateur qu'elle fait partie d'un système de gestion interne de restaurant. Cette approche permet de maintenir une expérience client fluide et centrée sur le service.

Mode Employé Assistant de gestion interne

Côté employé, l'assistant est accessible depuis le tableau de bord interne, réservé aux utilisateurs authentifiés. Dans ce mode, KEMI dispose d'un accès en lecture aux données de la base MySQL (produits, commandes, stocks, données RH) et peut :

Analyser l'état des stocks et signaler les ruptures imminentes ;

- Fournir des synthèses sur les commandes en cours et les performances de vente ;
- Répondre à des questions opérationnelles formulées en langage naturel (ex. : « Quels sont les plats les plus commandés cette semaine ? » ou « Combien d'employés sont en service aujourd'hui ? ») ;
- Assister le manager dans la prise de décisions quotidiennes.

Architecture technique

Le module IA est organisé en une application Django dédiée appelée `ia`, composée des fichiers suivants :

Table 11: fichiers dédiés du module intelligence artificielle

Fichier	Rôle
<code>ia/service.py</code>	Logique d'appel à l'API Anthropic, construction des prompts système selon le profil
<code>ia/views.py</code>	Endpoints Django recevant les messages et retournant les réponses au format JSON
<code>ia/urls.py</code>	Routage des deux endpoints : <code>/api/chat/client/</code> et <code>/api/chat/employee/</code>

La clé API est stockée de manière sécurisée dans le fichier `.env` du projet et chargée dans `settings.py` via la bibliothèque `python-decouple`, conformément aux bonnes pratiques de sécurité Django.

Sécurité

- L'endpoint `/api/chat/employee/` est protégé par le décorateur `@login_required` de Django : seuls les employés authentifiés peuvent y accéder ;
- L'endpoint `/api/chat/client/` est accessible sans authentification, afin de permettre une interaction naturelle avec les visiteurs ;
- La clé API Anthropic n'est jamais exposée côté client ni versionnée dans le dépôt GitHub.

10. CONCEPTION PRELIMINAIRE DE LA SOLUTION

10.1 Architecture technique retenue Le MVT de Django

L'application sera développée selon l'architecture MVT (Model – View – Template), propre au framework Django. Ce patron de conception est une adaptation du modèle MVC (Model – View – Controller) au contexte du développement web.

Table 12: présentation des couches du model MVT de Django

Couche	Composant Django	Rôle
Model	<code>models.py</code>	Définition des structures de données et interaction avec la base MySQL via l'ORM.
View	<code>views.py</code>	Logique métier, traitement des requêtes HTTP, préparation des données pour le template.
Template	<code>fichiers.html</code>	Présentation des données à l'utilisateur, interface graphique.
URL Routing	<code>urls.py</code>	Mapping des URLs vers les vues correspondantes.
Admin	<code>admin.py</code>	Interface d'administration personnalisable pour la gestion des données.

Ce tableau présente les différents fichiers (composant Django) qui seront obtenu par défaut (models.py, views.py, urls.py , admin.py) après l’installation de Django, que nous allons modifier pour la conception de notre application et des fichiers html que nous devons créer nous même pour la présentation des données a l’utilisateur, ainsi que le rôle de chacun de ces fichiers qui représente différentes couche de l’application.

Voici ce qui se passe exactement quand un utilisateur visite KMER_FOOD.com/produits/ :

1. Django reçoit la requête HTTP et consulte urls.py. Il trouve que le pattern /produits/ correspond à la vue liste_produits.
2. Django appelle la fonction liste_produits(request) dans views.py, en lui passant l'objet requête.
3. La vue exécute Produit.objects.all() — elle parle au Modèle dans models.py. Django traduit ça en SQL, interroge la base de données, et retourne une liste d'objets Produit.
4. La vue construit un dictionnaire contexte : {'produits': liste_de_produits} et appelle render(request, 'produits/liste.html', contexte).
5. Django charge le fichier produits/liste.html dans le dossier templates/, lui injecte le contexte, et exécute le moteur de gabarit. Chaque {{produit.nom}} est remplacé par la vraie valeur.
6. Django renvoie le HTML généré comme réponse HTTP au navigateur de l'utilisateur.

Stack technologique

- Langage : Python 3.12.3 ;
- Framework web : Django ;
- SGBD : MySQL (obligatoire selon le cahier des charges) ;
- ORM : Django ORM (aucune requête SQL brute) ;
- Environnement de développement : VSCode.
- Versionning : Github

10.2 Structure du projet Django

Le projet Django sera organisé en applications modulaires, chacune correspondant à un domaine fonctionnel. Cette approche favorise la séparation des responsabilités, la lisibilité et la maintenabilité du code.

Table 13: presentation des modules de l'application

Application Django	Modules couverts
accounts	Gestion des utilisateurs, authentification, rôles et permissions.

recipes	Gestion des recettes et associations recette-ingrédient.
products	Catalogue des plats, catégories et tarification.
orders	Commandes sur place et livraison, facturation.
inventory	Gestion des stocks, alertes et historique.
hr	Ressources humaines, affectations et paie.
dashboard	Tableau de bord, rapports financiers et statistiques.

10.3 Présentation du schéma de base de données

La base de données MySQL sur laquelle repose l'application a été préalablement conçue dans le cadre de travaux antérieurs. Elle couvre l'ensemble des entités nécessaires à la gestion du restaurant. Les principaux groupes de tables sont les suivants :

Gestion des utilisateurs et du personnel

Les tables dédiées aux utilisateurs stockent les informations d'authentification, les rôles, les permissions et les données personnelles des employés (fiches RH, affectations, éléments de paie).

Gestion culinaire

Ce groupe de tables décrit le catalogue de plats, les recettes associées, la liste des ingrédients et les quantités nécessaires à chaque préparation. Chaque recette référence le chef cuisinier responsable et le temps de cuisson prévu.

Gestion des commandes et de la facturation

Les tables de commandes enregistrent chaque transaction client, qu'il s'agisse d'une consommation sur place ou d'une livraison. Elles contiennent le détail des articles commandés, les quantités, les prix unitaires et le montant total calculé automatiquement.

Gestion des stocks

Les tables d'inventaire maintiennent l'état des stocks en temps réel, les mouvements d'entrée et de sortie, ainsi que les seuils d'alerte paramétrés pour chaque ingrédient.

Les modèles Django (fichiers models.py) seront construits en correspondance stricte avec ce schéma relationnel. L'ORM Django sera utilisé exclusivement pour interagir avec la base de données, garantissant ainsi la sécurité et la portabilité du code.

10.4 Stratégie de sécurité

La sécurité de l'application reposera sur plusieurs mécanismes complémentaires :

- Authentification Django native : gestion des sessions, hachage des mots de passe (PBKDF2 par défaut) ;
- Contrôle d'accès par rôle : chaque vue sera protégée par des décorateurs de permission (`@login_required`, `@permission_required`) ;
- Protection CSRF : activée par défaut dans Django sur tous les formulaires POST ;
- Protection contre les injections SQL : garantie par l'utilisation exclusive de l'ORM Django ;
- Journalisation : enregistrement des actions critiques (connexions, modifications sensibles) dans des fichiers de log.

10.5 Paradigme Orienté Objet appliqué au projet

Le développement de l'application repose sur le paradigme Orienté Objet (POO), qui constitue le fondement conceptuel du framework Django et du langage Python. Ce paradigme organise le logiciel autour d'objets — des entités autonomes regroupant des données (attributs) et des comportements (méthodes) — plutôt qu'autour de simples procédures.

a) Les principes fondamentaux de la POO appliqués au projet

Encapsulation

Chaque module fonctionnel de l'application est encapsulé dans une application Django distincte (accounts, recettes, produits, commandes, inventaire, hr, dashboard). Les données et la logique métiers propres à chaque module sont protégées et accessibles uniquement via des interfaces définies (vues, méthodes de modèles).

***Exemple :** Le calcul du montant total d'une commande est encapsulé dans une méthode `calculer_montant()` du modèle `Commande`, et non dispersé dans plusieurs vues.*

Héritage

Django exploite massivement l'héritage. Tous les modèles de l'application héritent de la classe de base `django.db.models.Model`, qui leur confère automatiquement les capacités d'interaction avec la base de données MySQL.

***Exemple :** La classe `Produit` hérite de `models.Model` et bénéficie ainsi de toutes les méthodes ORM (`save`, `delete`, `filter`, etc.).*

Polymorphisme

Le polymorphisme se manifeste dans la personnalisation des vues et des interfaces selon le rôle de l'utilisateur connecté. Une même URL peut retourner des contenus différents selon que l'utilisateur est un Directeur, un Serveur ou un Gestionnaire de stock.

Abstraction

L'ORM Django constitue une couche d'abstraction complète entre le code Python et la base de données MySQL. Le développeur manipule des objets Python sans écrire de requêtes SQL brutes, ce qui simplifie le développement et garantit la sécurité.

b) Modélisation Orientée Objet — Diagrammes UML

Conformément aux bonnes pratiques du génie logiciel, la conception Orientée Objet de l'application sera formalisée à travers quatre diagrammes UML :

Table 14: objectif des diagrammes UML

Diagramme	Outil utilisé	Objectif
Diagramme de Classes	StarUML	Représenter les entités du système, leurs attributs, méthodes et relations.
Diagramme de Cas d'Utilisation	StarUML	Identifier les acteurs et les fonctionnalités accessibles à chacun.
Diagramme de Séquence	StarUML	Décrire les interactions entre objets pour les scénarios clés.
Diagramme de Déploiement	StarUML	Représenter l'architecture physique de déploiement de l'application.

Ce tableau présente les objectifs de chaque diagramme dans la conception de cette application, et le logiciel utilisé pour leur modélisation. Ces diagrammes seront présentés en détail dans les sections suivantes du rapport.

c) Structure modulaire Orientée Objet du projet

L'organisation du projet Django en applications modulaires reflète directement le découpage Orienté Objet du système :

Table 15: Structure modulaire Orientée Objet du projet

Application Django	Classe principale	Relations
accounts	Utilisateur	Hérite de AbstractUser (Django)
recipes	Recette, Ingrédient	Recette ↔ Ingrédient (ManyToMany)
products	Produit, Catégorie	Produit → Catégorie (ForeignKey)
orders	Commande, LigneCommande	Commande ↔ Produit (ManyToMany)
inventory	Stock, MouvementStock	Stock → Ingrédient (ForeignKey)
hr	Employe, Affectation	Employé → Utilisateur (OneToOne)
dashboard	—	Agrégation de données des autres modules

11. PLANIFICATION ET ORGANISATION DU TRAVAIL

11.1 Méthodologie Agile

Le groupe adoptera une méthodologie Agile simplifiée, recommandée par le cahier des charges. Le projet sera découpé en sprints d'une semaine, chacun

ciblant un ou plusieurs modules fonctionnels. Des réunions de synchronisation hebdomadaires permettront de suivre l'avancement, d'identifier les blocages et d'ajuster le plan si nécessaire.

11.1.1 Planning prévisionnel

Table 16: planning Agile

Sprint	Travaux prévus	Livrables attendus
S1	Mise en place de l'environnement de développement, configuration Django et MySQL, création du repository GitHub.	Projet Django initialisé, connexion BD établie.
S1	Définition de tous les modèles Django (models.py) et migrations.	Schéma BD Django opérationnel.
S2	Module Utilisateurs : authentification, rôles, interface d'administration.	Connexion et gestion des accès fonctionnels.
S3	Modules Recettes et Produits : catalogue, catégories, tarification.	CRUD Recettes et Produits.
S4	Module Commandes : prise de commande, calcul montant, facturation.	Workflow de commande opérationnel.
S5	Modules Inventaire et RH : stocks, alertes, dossiers employés, paie.	Gestion des stocks et du personnel.
S5	Tableau de bord : indicateurs financiers, statistiques, rapports dynamiques.	Dashboard complet.
S6	Tests fonctionnels globaux, corrections,	Application finale, rapport
	finalisation du code et documentation.	technique, manuel utilisateur.

Ceci est le plan de notre travail d'après la méthode agile .

11.2 Module 1: AUTHENTIFICATION

Le module d'authentification a été conçu pour être discret : le client ne peut pas distinguer au premier abord s'il s'agit d'une interface de marketing digital ou d'une application de gestion. L'authentification s'effectue via l'adresse e-mail (e-mail d'entreprise pour les employés, e-mail usuel pour les clients) et un mot de passe.

Les données de la base MySQL préexistante ont été importées dans Django. Les connexions avec les comptes existants (employés et clients) sont fonctionnelles. La gestion des cas d'erreur (e-mail inexistant, mot de passe erroné) est en cours de finalisation.

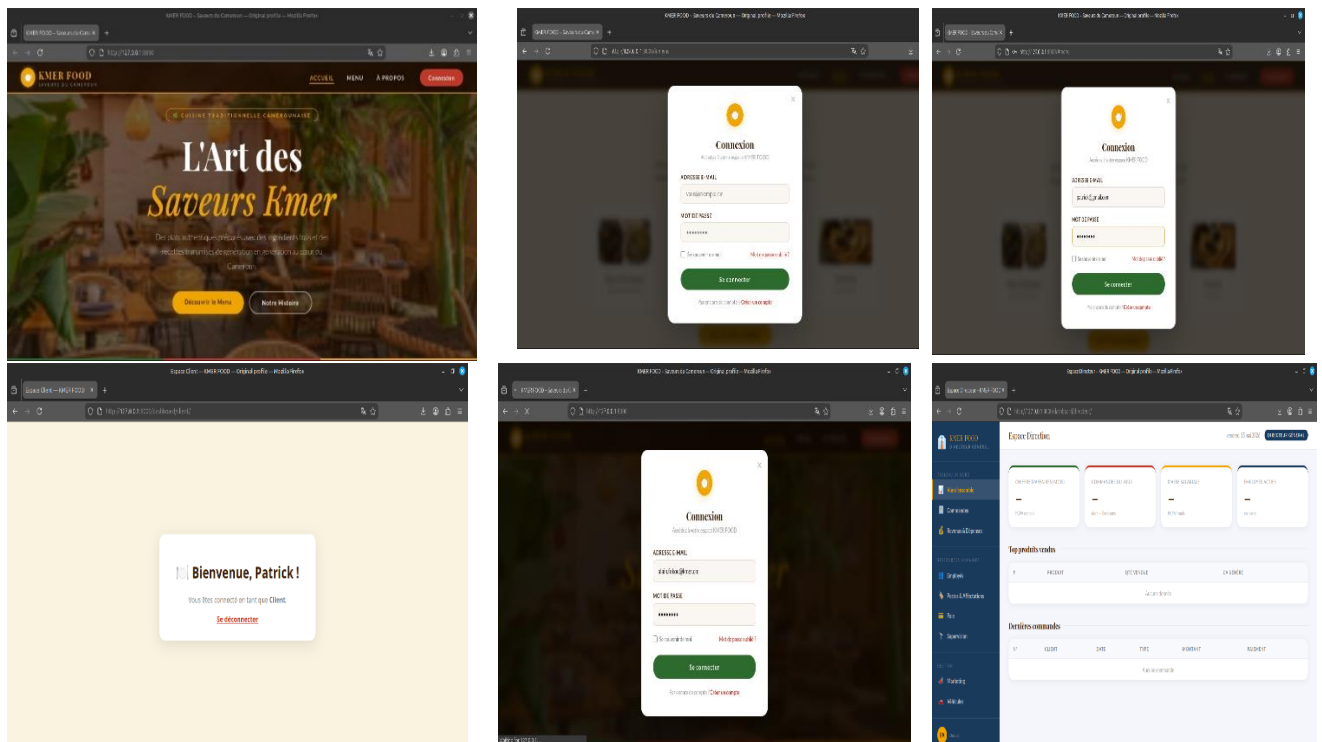


Figure 16: exemple de suivi d'une connexion avec des comptes déjà existants de la base de données

11.3 Module 2 :RECETTES

Le module de gestion des recettes constitue le cœur culinaire de l'application. Il a été conçu pour que le chef cuisinier dispose d'un espace de travail complet et intuitif, lui permettant de centraliser l'ensemble du savoir-faire culinaire du restaurant dans un catalogue numérique structuré.

Lors de la création d'une recette, le chef renseigne le nom du plat, le temps de cuisson estimé, les instructions de préparation, puis associe chaque ingrédient nécessaire avec sa quantité précise. Cette association est gérée en base de données via la table intermédiaire RecetteIngrédient, qui matérialise la relation ManyToMany entre Recette et Ingrédient. Django valide automatiquement qu'aucune recette ne peut être enregistrée sans au moins un ingrédient associé — une règle métier appliquée au niveau du formulaire et du modèle.

Les données des recettes et des ingrédients préalablement définies dans la base MySQL existante ont été importées dans l'application. Le chef cuisinier peut désormais consulter l'ensemble du catalogue existant, modifier les quantités d'ingrédients en fonction des ajustements de la cuisine, et ajouter de nouvelles recettes selon l'évolution de la carte. La suppression d'une recette est bloquée par une contrainte d'intégrité Django si cette recette est encore référencée par un plat actif du catalogue — évitant ainsi toute incohérence dans le système. La gestion des droits garantit que les cuisiniers disposent d'un accès en lecture seule : ils consultent les recettes pour exécuter les préparations, mais ne peuvent ni créer ni modifier.

11.2 Module 3: PRODUITS

Le module des produits représente la vitrine numérique du restaurant. Il gère l'ensemble du catalogue des plats proposés à la clientèle, depuis leur création jusqu'à leur mise en vente.

Chaque produit est défini par un nom, une description détaillée, une photo, un prix de vente, et une catégorie parmi les quatre définies : entrées, plats principaux, desserts et boissons. La catégorisation est obligatoire — Django rejette tout enregistrement sans catégorie associée. La tarification est dynamique : le Directeur ou le

Chef cuisinier peuvent modifier le prix d'un plat à tout moment depuis leur interface dédiée, avec une validation qui empêche la saisie d'un prix nul ou négatif.

La règle métier la plus critique de ce module est la condition de mise en vente : un plat ne peut apparaître dans le catalogue visible par les clients que s'il est associé à une recette valide et active dans le système. Cette contrainte crée un lien fort entre le module Produits et le module Recettes, garantissant que rien n'est proposé à la vente sans que la cuisine puisse en assurer la préparation. Les données du catalogue existant ont été importées depuis la base MySQL et sont consultables depuis l'interface d'administration Django. La mise en forme de l'interface cliente est en cours de développement.

11.2 Module 4 :COMMANDES ET FACTURATION

Le module de gestion des commandes est la pièce centrale du flux opérationnel de l'application. Il orchestre l'ensemble du cycle de vie d'une transaction client, depuis la prise de commande jusqu'à la génération de la facture.

Lorsqu'un serveur initie une nouvelle commande,, chaque produit sélectionné génère automatiquement une LigneCommande entité en composition avec la commande parente, ce qui signifie qu'une ligne de commande ne peut exister sans la commande à laquelle elle appartient. Le montant total est calculé automatiquement par la méthode `calculer_montant()` du modèle Commande, en multipliant le prix unitaire de chaque produit par la quantité commandée, puis en sommant l'ensemble des lignes.

Le système suit l'état de chaque commande à travers quatre statuts successifs : **en attente**, **en préparation**, **livrée**, **annulée**. Seule une commande encore à l'état « en attente » peut être annulée une contrainte appliquée au niveau de la vue Django pour éviter toute annulation d'une commande déjà en cuisine. Dès qu'une commande passe au statut « en préparation », elle déclenche automatiquement la mise à jour du stock : les quantités de chaque ingrédient correspondant aux plats commandés sont décrémentées dans le module Inventaire. La facture est générée automatiquement à la validation de la commande et peut être imprimée ou transmise au client. L'application supporte également les commandes en livraison, distinguées des commandes sur place par un champ de type dans le modèle.

11.2 Module 5 : INVENTAIRE ET ALERTES STOCK

Le module de gestion de l'inventaire assure la traçabilité complète des stocks d'ingrédients en temps réel. Son rôle est double : réagir automatiquement aux événements du système et permettre au gestionnaire de stock d'anticiper les ruptures.

Chaque ingrédient du catalogue est associé à une entrée dans la table Stock, qui maintient la quantité disponible en temps réel et un seuil critique paramétrable. À chaque validation de commande, le module Commandes communique avec l'Inventaire via une mise à jour automatique des quantités : pour chaque ligne de commande, Django calcule la consommation d'ingrédients correspondante à partir des recettes associées aux produits commandés, puis décrémente les stocks en conséquence. Ce décrément est enregistré dans la table MouvementStock, qui constitue l'historique complet de toutes les entrées et sorties.

La méthode `verifier_seuil()` est appelée automatiquement après chaque décrément. Si la quantité restante d'un ingrédient passe en dessous du seuil critique défini, une alerte est générée et notifiée au gestionnaire de stock depuis son tableau de bord. Aucune commande ne peut être validée si un ingrédient nécessaire est en rupture totale — une vérification préalable bloque la transaction avant même la création des lignes de commande. Le gestionnaire de stock est également en mesure d'enregistrer manuellement des entrées en stock lors des réapprovisionnements, ce qui génère à son tour un MouvementStock de type « entrée ». Seul le gestionnaire de stock est habilité à modifier les seuils critiques des ingrédients.

11.2 Module 6 : RH ET PAIE

Le module des ressources humaines centralise l'ensemble des informations relatives au personnel du restaurant et automatise les processus administratifs les plus récurrents.

Chaque employé dispose d'une fiche complète dans le système, comprenant ses informations personnelles, son poste, la nature de son contrat et sa date d'embauche. Cette fiche RH est liée au compte utilisateur de l'employé par une relation OneToOne : la classe `Employe` et la classe `Utilisateur` sont deux entités distinctes, mais chaque employé correspond exactement à un compte d'accès dans l'application. Cette séparation garantit une flexibilité architecturale — les informations d'authentification et les données RH sont gérées indépendamment, mais restent cohérentes.

L'affectation des employés aux différents services (salle, cuisine, accueil, stock) est gérée dynamiquement : un employé peut être réaffecté à un autre service sans que cela n'impacte son dossier contractuel. Le calcul de la paie est automatisé par la méthode `calculer_paie()` du modèle `Personnel`, qui prend en compte le salaire de base associé au poste et le nombre de jours travaillés enregistrés dans le mois. Seul l'Administrateur est autorisé à modifier les informations contractuelles (salaire de base, type de contrat) d'un employé, tandis que le Directeur dispose d'un accès en lecture pour superviser les données globales de la masse salariale depuis le tableau de bord.

11.7 Module 7: TABLEAU DE BORD ET RAPPORT

Le tableau de bord est le module de pilotage stratégique de l'application. À la différence des six autres modules, il ne possède pas de modèle Django propre : il agrège les données en temps réel depuis l'ensemble des modules pour produire des indicateurs synthétiques à destination du Directeur et de l'Administrateur.

À l'ouverture du tableau de bord, quatre blocs d'indicateurs clés sont affichés : le chiffre d'affaires du mois en cours (calculé par la méthode `ca_mensuel()` qui somme les montants de toutes les commandes livrées du mois), le nombre total de commandes traitées, le classement des cinq produits les plus vendus sur la période sélectionnée (`top_produits()`), et le total des dépenses d'approvisionnement enregistrées par le gestionnaire de stock. Ces données sont actualisées à chaque chargement de page sans nécessiter de rechargement manuel.

Le Directeur peut ajuster la période d'analyse — vue journalière, hebdomadaire ou mensuelle — pour obtenir une granularité adaptée à ses besoins de décision. Les rapports financiers peuvent être exportés au format PDF directement depuis l'interface, permettant leur archivage ou leur présentation lors des réunions de direction. L'accès à ce module est strictement réservé au Directeur et à l'Administrateur, contrôlé par le décorateur `@permission_required` appliqué à chaque vue du tableau de bord. Aucun autre rôle, y compris le Chef cuisinier ou le Gestionnaire de stock, ne peut accéder à ces données financières consolidées.

Table 17: Recapitulatif des livrables

Livrable	Statut	Responsable
Environnement Python/Django configuré	✓ Complété	Toute l'équipe
Installation StarUML, Anaconda	✓ Complété	Toute l'équipe
Installation Git et Github	✓ Complété	Toute l'équipe
Base MySQL créée et connectée	✓ Complété	Toute l'équipe
Dépôt GitHub initialisé et structuré	✓ Complété	Toute l'équipe

Diagramme de Gantt (GanttProject)	✓ Complété	Vmessah Ida
Diagramme de cas d'utilisation (StarUML)	✓ Complété	Toute l'équipe
Diagramme de classes (StarUML)	✓ Complété	Vmessah Ida
Diagrammes de séquence (StarUML)	✓ Complété	Toute l'équipe
Diagramme de déploiement (StarUML)	✓ Complété	Engamba Armand
Module 1	✓ Complété	Vmessah Ida
Module 2	✓ Complété	Engamba Armand
Module 3	✓ Complété	Engamba Armand
Module 4	✓ Complété	Vmessah Ida
Module 5	✓ Complété	Engamba Armand
Module 6	✓ Complété	Vmessah Ida
Module 7	✓ Complété	Engamba Armand
Tests fonctionnels	En cours	Vmessah & Engamba

Ici il s'agit du récapitulatif des travaux déjà fait ceux en cours et ceux à faire, avec attribution des responsables qui aurait dû se faire selon notre diagramme de GANTT, mais qui ne sera pas respecté.

12. Difficultés rencontrées et solutions apportées

Ce projet a été l'occasion de confronter les connaissances théoriques aux réalités du développement logiciel en équipe. Les difficultés rencontrées, loin d'être des obstacles, ont constitué autant d'opportunités d'apprentissage approfondi.

Au cours de ce projet nous avons fait face à certaines difficultés parmi lesquelles :

Table 18: difficultés rencontrées et solution apportées

Difficulté rencontrée	Impact	Solution apportée
Installation des logiciels requis (MySQL, GanttProject, StarUML)	Perte de temps en début de projet	Procédure d'installation documentée et partagée dans le README du dépôt GitHub.
Prise en main de GitHub — gestion des branches et des conflits	Risque de perte de code ou de conflits non résolus	Formation par tutoriels vidéo ; adoption du feature-branch workflow ; règle : un membre valide les Pull Requests d'un autre.
Connexion MySQL ↔ Django — erreurs de configuration DATABASES	Blocage du démarrage du projet	Diagnostic par les messages d'erreur Django ; utilisation de python-

		dotenv pour sécuriser les identifiants.
Configuration des applications Django (INSTALLED_APPS, migrations)	Modèles non reconnus par l'ORM	Rigueur dans la procédure : startapp → ajout dans INSTALLED_APPS → makemigrations → migrate.
Compréhension du pattern MVT et séparation des responsabilités	Logique métier parfois mélangée avec les templates	Relecture de la documentation officielle Django ; revue de code entre membres.
Importation des données existantes de MySQL vers Django	Tables existantes non reconnues par les modèles Django	Utilisation de inspectdb pour générer les modèles depuis le schéma existant, puis adaptation manuelle.
La deconnexion après l'authentification est difficile pour l'instant		

13. CONCLUSION ET PERSPECTIVES

13.1 Bilan du rapport de lancement

Ce rapport de lancement constitue le cadre de référence qui guidera l'ensemble du développement de l'application KMER FOOD. Il pose les fondements théoriques, fonctionnels, architecturaux et organisationnels du projet, en s'appuyant sur une analyse rigoureuse du domaine métier et des exigences du cahier des charges.

L'analyse des besoins a mis en évidence sept modules fonctionnels interdépendants, à ces sept modules s'ajoute un huitième composant, développé en dehors du cahier des charges initial : l'assistant intelligent KEMI, fondé sur l'API Anthropic, qui enrichit l'expérience client par des conseils nutritionnels personnalisés et assiste les employés dans la gestion opérationnelle quotidienne du restaurant, dont la réalisation nécessite une maîtrise de l'architecture MVT de Django, de la modélisation relationnelle MySQL, et des bonnes pratiques de développement web sécurisé. La conception préliminaire organisation en applications Django modulaires, schéma de base de données structuré, stratégie de sécurité multicouche offre une base solide pour la phase d'implémentation.

13.2 Impacts attendus

À terme, l'application KMER FOOD vise à digitaliser les cinq processus métiers clés d'un restaurant camerounais :

- 1 Réduction estimée de 80 % des erreurs manuelles de commande et de saisie de stock.
- 2 Traçabilité complète des opérations critiques grâce à la journalisation.
- 3 Visibilité financière en temps réel pour le Directeur via le tableau de bord analytique.
- 4 Fluidification de la communication entre acteurs (serveurs, cuisiniers, gestionnaire de stock).
- 5 Alignement sur les standards de l'industrie (architecture MVT, ORM, sécurité CSRF/PBKDF2).

13.3 Perspectives d'évolution

Ce projet, conçu avec une architecture modulaire et scalable, offre plusieurs axes d'évolution à moyen terme :

- 6 Déploiement en production sur un serveur VPS Linux avec Nginx et Gunicorn.
- 7 Développement d'une application mobile complémentaire via une API REST Django REST Framework.
- 8 Intégration d'un système de paiement mobile (Mobile Money — Orange Money, MTN MoMo).
- 9 Extension du module dashboard avec des indicateurs prédictifs (Machine Learning).

10 Adaptation du système pour d'autres établissements de restauration camerounais.

14. Références

1. Cahier des charges du projet — M. Mbietieu Amos Mb., ENSPY, 2025-2026.

2. https://github.com/Vmesssah-Ida/KMER_FOOD_RESTO_GROUPE4 (le lien du depot github)

3. <https://youtu.be/xwFj6WCJW0I?si=rvAxyK3k6KJJXoy> (comment utiliser Github)

4. <https://youtu.be/fKUn5f-kSAg?si=pxLuJWmtT3UovykR> (diagramme de sequence et les autres diagrammes)

5. https://youtu.be/Bn0k9DDYBZM?si=u9_blxu6EHTCrm5Q (formation partielle sur django)

6. Anthropic. *Documentation officielle de l'API Claude — SDK Python.*
<https://docs.anthropic.com>
